

Time-Series Patterns

 systemdesignschool.io/fundamentals/time-series-patterns



The previous article showed how materialized views pre-compute aggregates for instant queries. But those aggregates are themselves time-series data—metrics measured at regular intervals. As metrics accumulate, storage grows without bound. Querying months of history gets slower.

This article covers patterns for efficient time-series storage: bucketing, rollups, and retention policies.

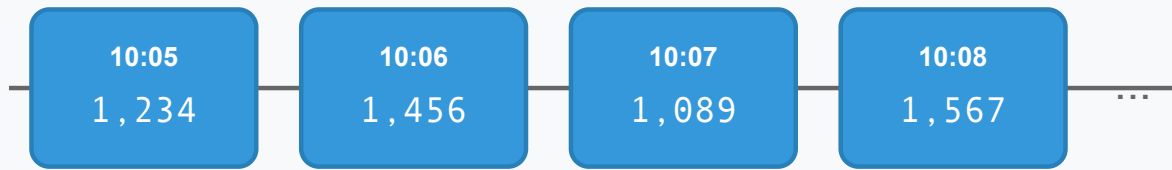
Bucketing: Fixed Time Boundaries

The simplest pattern stores aggregates in time-aligned buckets. The key includes the time boundary:

```
metric:page_views:minute:2024-01-15T10:05:00
metric:page_views:minute:2024-01-15T10:06:00
metric:page_views:minute:2024-01-15T10:07:00
```

Why fixed boundaries? Consider querying "views in the last 10 minutes." With minute-aligned buckets, this fetches exactly 10 keys. No scanning. No filtering within buckets. The query planner knows exactly which keys to read before fetching any data.

Time Bucketing



`page_views:minute:2024-01-15T10:05:00 → 1234`

Fixed time boundaries. Query "last 10 min" = fetch exactly 10 keys.



Without fixed boundaries—storing events at arbitrary timestamps—querying a time range requires scanning and filtering. Fixed boundaries convert range queries into point lookups.

Choosing Bucket Granularity

Match bucket size to how the data will be queried:

Granularity	Use Case
-------------	----------

1 second	Real-time monitoring dashboards, rapid feedback loops
----------	---

1 minute	Operational dashboards, alerting systems
----------	--

1 hour	Business metrics, daily pattern analysis
--------	--

1 day	Historical trends, weekly/monthly reports
-------	---

Finer granularity gives more detail but costs more storage. A metric with 1-second resolution produces 86,400 data points per day. At 1-minute resolution, that drops to 1,440.

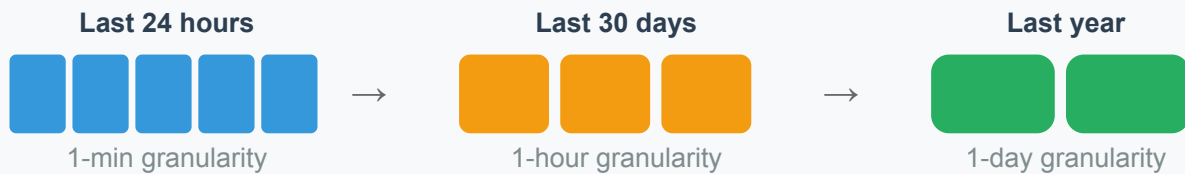
Checkpoint: Bucketing solves the query efficiency problem. But storing fine-grained data indefinitely isn't practical. A year of 1-second data is 31 million points per metric. How do we keep recent data detailed while compacting older data?

Rollups: Coarser Granularity Over Time

Users need second-level detail for live dashboards. But nobody zooms into a one-second window from six months ago. Match retention granularity to query patterns.

A **rollup** aggregates fine-grained buckets into coarser ones. As data ages, detail decreases but storage stays bounded.

Rollup Pattern



60 one-minute buckets → 1 hourly bucket → 24 hourly → 1 daily
Storage drops from 31M points/year to roughly 10K points/year



A typical retention policy:

- **Last 24 hours:** 1-minute granularity (1,440 points per metric)
- **Last 30 days:** 1-hour granularity (720 points per metric)
- **Last year:** 1-day granularity (365 points per metric)

Total storage: roughly 2,500 points per metric per year, regardless of how many years of data you keep. Without rollups, one year of 1-second data would be 31 million points.

How Rollups Work

A background job runs periodically to aggregate older data:

1. Read the 60 one-minute buckets for 10:00-10:59
2. Compute aggregates (sum, average, min, max, count) into one 1-hour bucket
3. Write the 1-hour bucket
4. Delete or expire the minute-level buckets

The rollup job typically runs hourly for minute-to-hour aggregation, and daily for hour-to-day aggregation. Timing matters: run after all events for the period have arrived (accounting for late data).

What aggregates to keep depends on your queries. For metrics you'll sum (total requests), keep the sum. For metrics you'll average (latency), keep both sum and count so you can compute accurate averages across rolled-up periods. Min and max are useful for alerting ("what was the peak?").

Retention Policies with TTL

Many time-series stores support TTL (time-to-live)—keys automatically expire after a configured period.

Set TTLs to match your retention tiers:

- Minute-level buckets: 24-hour TTL

- Hour-level buckets: 30-day TTL
- Day-level buckets: 1-year TTL or indefinite

TTL-based expiration handles cleanup without explicit deletion jobs. Redis, many time-series databases, and some NoSQL stores support this natively.

Practical tip: Match retention to actual query patterns. If nobody queries minute-level data older than 2 hours, reduce the TTL. Storage costs drop with no impact on users. Analyze your query logs to find the right boundaries.

Handling High-Cardinality Dimensions

Rollups work well when the number of distinct keys is bounded. But what about high-cardinality dimensions?

Consider tracking "page views by user." With millions of users, you'd need millions of keys per time bucket. Storage explodes. Most of those keys have counts of 1 or 2—not useful for dashboards.

For high-cardinality cases, approximate algorithms help:

HyperLogLog for distinct counts. "How many unique users visited today?" The answer might be 1,847,293 or 1,852,106—either is useful for dashboards. HyperLogLog provides estimates within a few percent accuracy using fixed memory (typically 12 KB), regardless of cardinality.

Count-Min Sketch for frequency estimation. "How often did each URL appear?" A probabilistic data structure tracks approximate counts without storing every distinct value.

Sampling for high-volume streams. A 1% sample of events provides statistically valid aggregates for large-scale patterns. Store 1% of events in full detail, use aggregates for the rest.

These approximations trade precision for bounded storage. For most analytics use cases, "roughly 1.8 million users" is as useful as "exactly 1,847,293 users."

What's Next

Time-series patterns solve the storage efficiency problem. The remaining question is how to put these pieces together—stream processors, materialized views, and time-series storage—into a working analytics system with patterns for different latency and accuracy requirements.